

TensorBinding.jl

Package Function Reference

May 5, 2026

Contents

1	Overview	4
2	Central Data Structure: TBHamiltonian	5
3	Hamiltonian Construction (TBSystem.jl)	5
3.1	Constructor	5
3.2	Cache and Truncation Helpers	6
3.3	Incremental Hamiltonian Modification	7
3.4	Geometry Helpers	7
4	Low-Level MPO Builders (Hamiltonian.jl, 2D_lattice.jl)	8
4.1	1D Kinetic Operators	8
4.2	2D Kinetic Operators	9
4.3	Diagonal Mask MPOs	9
4.4	QTCI-Based Builders	10
4.5	Preset Model Hamiltonians (direct MPO)	10
5	Generic nth-Neighbor Hopping (nnneighbor_tk.jl)	11
5.1	Shell Detection	11
5.2	Hopping Accumulator	11
6	Bilayer and Multilayer Hamiltonians	12
6.1	Commensurate Stacking (bilayer_tk.jl)	12
6.2	Twisted Multilayer (twisted_tk.jl)	13
6.3	Layer-Core Prepend/Postpend Helpers	13
7	Spin and Nambu Extensions (Supercond_tk.jl)	13
7.1	Index Constructors	13
7.2	Unified Prepend/Postpend Operators (utils.jl)	14
7.3	Spin and Nambu Wrappers	14
7.4	Higher-Level Assemblers	14
8	Kernel Polynomial Method (KPM_tk.jl)	14
8.1	Spectral Bounds Estimation	15
8.2	Building the Chebyshev List	15
8.3	Spectral Observables from Stored Chebyshev List	16
8.4	High-Level LDOS Dispatch (TBHamiltonian interface)	16
8.5	HODC Kernel	17

9	Density Matrix Purification (Purification.tk.jl)	17
9.1	Initial Guess	17
9.2	McWeeny Purification	17
9.3	SP2 Purification	18
10	Quantum Fourier Transform and Band Structure (QFT.tk.jl)	18
10.1	QFT Conjugation	18
10.2	Online Band Structure: <code>get_bands</code>	18
10.3	High-Symmetry k-Path Utilities	19
10.4	Auxiliary Index Projection Helpers	19
11	Topological Invariants (Topology.tk.jl)	20
11.1	Projector Retrieval	20
11.2	Quenched Position Operators	20
11.3	1D Winding Number	21
11.4	2D Real-Space Chern Marker	21
11.5	Thouless Pump	22
12	DMRG (DMRG.tk.jl)	22
13	Time Evolution (Timeev.tk.jl)	23
13.1	Propagator MPO and Single-Step TDVP	23
13.2	Trajectory Evolution: MPS States	23
13.3	State Analysis Utilities	23
13.4	Density Matrix Dynamics (open / driven systems)	24
13.5	Observables Along Trajectories	24
13.6	Diagnostic Tools	25
14	Two-Particle Systems (twoparticle.tk.jl)	25
14.1	High-Level Constructors	25
14.2	Low-Level MPO Builders	26
15	Random Phase Approximation (RPA.tk.jl)	26
15.1	Pipeline	26
15.2	Functions	26
15.3	Tensor Utilities (used internally by RPA)	27
16	Self-Consistent Mean Field (Meanfi.tk.jl)	27
17	Utilities (utils.jl)	27
18	Custom Operator Extensions	28
19	Quick-Reference Workflows	29
19.1	KPM Density of States and Band Structure	29
19.2	Density Matrix Purification	29
19.3	Topological Invariants	29
19.4	Bilayer and Twisted Multilayer	30
19.5	Spinful Superconductor (BdG)	30
19.6	Band Structure with Sublattice Resolution	30
19.7	<code>aux_side</code> : postpending auxiliary DOF	31
20	Krylov Green's Function (krylov.tk.jl)	31

21 Flake Masking (Flake_tk.jl)	32
21.1 SDF Primitives	32
21.2 CSG Combinators	32
21.3 Mask Application	32

Overview

TensorBinding.jl is a Julia package for constructing and manipulating tight-binding Hamiltonians as Matrix Product Operators (MPOs) in the *quantics binary* (QTT) representation. A system of $N = 2^L$ sites is encoded in L qubit sites: site index $n \in \{0, \dots, N-1\}$ is identified with the binary string $b_1 b_2 \dots b_L$. This representation makes the bond dimension of many physically relevant Hamiltonians small (typically ≤ 10), enabling compression via Quantics Tensor Cross Interpolation (QTCI).

Key capabilities:

- MPO construction for 1D and 2D lattices (square, triangular, triangular Bravais, honeycomb, kagomé, Lieb, dice) via shift operators and QTCI. Sublattice-aware models carry an explicit sublattice index and exact real-space positions.
- Generic n th-nearest-neighbor hopping via `add_hopping_2D!`: uniform, direction-dependent, site-dependent, or fully position+direction-dependent amplitude functions on any 2D geometry.
- Bilayer, multilayer, twisted, and Bernal-stacked Hamiltonians.
- Spin- $\frac{1}{2}$ and Nambu (BdG) extensions via a prepend/postpend-core pattern, with auxiliary DOF placeable at front (`:pre`) or back (`:post`).
- Kernel Polynomial Method (KPM) for density of states, density matrices, spectral functions, and Green's functions.
- Density-matrix purification (McWeeny, SP2) with `TBHamiltonian`-aware API.
- Quantum Fourier Transform conjugation for band structure $A(k, \omega)$, supporting spinful, BdG, and layered Hamiltonians via `aux_proj`.
- Topological invariants: real-space Chern marker and winding-number density via quenched or flat position operators, compatible with any lattice geometry.
- DMRG ground state and spectral DMRG.
- Real-time evolution: TDVP (fixed and time-dependent H), propagator MPO.
- Random Phase Approximation (RPA) for susceptibility.
- Self-consistent mean-field (Hubbard model).

Site encoding conventions:

- *Position qubits*: L sites, site i (0-indexed) $\leftrightarrow$ binary string $b_1 \dots b_L$, big-endian.
- *Extended systems*: auxiliary indices can be *prepended* (`:pre`, default) or *postpended* (`:post`). For a spinful bilayer BdG system with `:pre` the site chain is `[nambu_s, spin_s, layer_s, pos_qubits...]`.
- *2D lattices*: row-major flattening, linear index $i = i_x + i_y \cdot 2^{L_x}$, $i_x \in \{0, \dots, 2^{L_x} - 1\}$, $i_y \in \{0, \dots, 2^{L_y} - 1\}$.
- *Honeycomb (legacy "hex_2d")*: intra-row bonds pair even and odd i_x sites; two inter-row bond types controlled by sublattice masks. Requires $L = L_x + L_y$ total qubits.
- *Sublattice-explicit models* ("`honeycomb`", "`honeycomb_nnn`", "`kagome`", "`lieb`", "`dice`"): use a triangular Bravais basis of $2^{L_x} \times 2^{L_y}$ unit cells ($L = L_x + L_y$ qubits) plus one postpended sublattice index of dimension $n_{\text{sub}} \in \{2, 3\}$. Total physical sites = $n_{\text{sub}} \cdot 2^L$; H.N counts unit cells.

Central Data Structure: TBHamiltonian

Defined in `src/TBSystem.jl`.

Field	Type	Description
<code>L</code>	<code>Int</code>	Number of position qubits; $N = 2^L$ sites.
<code>N</code>	<code>Int</code>	Number of physical sites per spin/Nambu/layer sector.
<code>sites</code>	<code>Vector{<:Index}</code>	All ITensor site indices (includes auxiliary indices if present).
<code>mpo</code>	<code>MPO</code>	The accumulated Hamiltonian MPO.
<code>geometry</code>	<code>Function</code> or <code>Nothing</code>	Callable <code>i -> pos_vec</code> (1-indexed); returns real-space position of site <code>i</code> .
<code>scale</code>	<code>Float64</code>	Energy half-bandwidth; $H/scale$ has spectrum $\subseteq [-1, 1]$. Set to 0 for lazy DMRG estimation.
<code>center</code>	<code>Float64</code>	Spectral center $(E_{\max} + E_{\min})/2$. 0 for particle-hole symmetric spectra.
<code>spin_s</code>	<code>Index</code> or <code>Nothing</code>	Spin- $\frac{1}{2}$ index (set by <code>add_spin!</code>).
<code>nambu_s</code>	<code>Index</code> or <code>Nothing</code>	Nambu index (set by <code>add_superconductivity!</code>).
<code>layer_s</code>	<code>Index</code> or <code>Nothing</code>	Layer index (set by bilayer/multilayer constructors).
<code>sublattice_s</code>	<code>Index</code> or <code>Nothing</code>	Sublattice index postpended by sublattice-aware constructors (honeycomb, kagomé, Lieb, dice).
<code>aux_side</code>	<code>Symbol</code>	<code>:pre</code> (aux at front, default) or <code>:post</code> (aux at back).
<code>_tn_cache</code>	<code>Vector{MPO}</code> or <code>Nothing</code>	Cached MPO Chebyshev list $T_0 \dots T_N$ (mode <code>:mpo</code>).
<code>_tn_mps_cache</code>	<code>Vector{MPS}</code> or <code>Nothing</code>	Cached MPS Chebyshev list (mode <code>:mps</code>); set by <code>KPM_Tn</code> with <code>mode=:mps</code> .
<code>_tn_Ncheb</code>	<code>Int</code>	Order N of cached Chebyshev list (shared by both caches).
<code>_density_cache</code>	<code>MPO</code> or <code>Nothing</code>	Cached density matrix from purification.

Do not construct directly; use `get_Hamiltonian` or a layer/multilayer constructor. Caches are invalidated automatically by all `add_*` functions and `truncate!`. The `center` field is set together with `scale` during spectral-bounds estimation and is needed to correctly rescale energies: $\tilde{\omega} = (\omega - \text{center})/\text{scale}$.

Hamiltonian Construction (TBSystem.jl)

Constructor

Signature	Description
-----------	-------------

<code>get_Hamiltonian(geometry, params; L, scale, tol, maxdim, kwargs)</code>	Build a <code>TBHamiltonian</code> from a named model. <code>geometry</code> is a string key; <code>params</code> holds model parameters (see below). Returns a <code>TBHamiltonian</code> .
---	---

Supported geometry strings:

geometry	params	Notes
"chain_1d"	hopping t	1D NN chain, direct MPO (no QTCI).
"square_2d"	hopping t	2D square, $2^{L_x} \times 2^{L_y}$ sites; <code>Lx/Ly</code> kwargs.
"hex_2d"	hopping t	2D honeycomb.
"triangular_2d"	hopping t	2D triangular.
"ssh"	(t, d)	SSH chain: hopping $t \pm d$ on alternating bonds.
"aah"	(V, ϕ, t)	Aubry–André–Harper quasicrystal.
"uniform"	t	Uniform 1D with optional on-site v .
"haldane"	(t_2, ϕ, M) <code>NamedTuple</code>	Haldane model; requires <code>rs</code> ($N \times 2$ position matrix).
"chernhex"	(t, t_2, ms)	Haldane-type Chern insulator on honeycomb.
"chern8"	(V, t)	8-fold Chern mosaic on square lattice.
"qc2dsquare"	t	Quasicrystal-modulated square lattice.
"honeycomb"	$(t,)$ <code>NamedTuple</code>	Honeycomb with explicit 2-atom sublattice index (postpended); requires <code>Lx/Ly</code> kwargs.
"honeycomb_nnn"	(t, t_2) <code>NamedTuple</code>	Honeycomb with NN hopping t and NNN same-sublattice hopping t_2 .
"kagome"	$(t,)$ <code>NamedTuple</code>	Kagomé lattice, 3-atom unit cell; <code>Lx/Ly</code> required.
"lieb"	$(t,)$ <code>NamedTuple</code>	Lieb lattice, 3-atom unit cell; flat band at $E = 0$.
"dice"	$(t,)$ <code>NamedTuple</code>	Dice (T3) lattice, 3-atom unit cell (hub A, rim B/C).
"triangular_bravais"	hopping t	Bravais triangular lattice: $\mathbf{a}_1 = (1, 0)$, $\mathbf{a}_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, three clean bond types; <code>Lx/Ly</code> required.
"custom"	$f(i, j)$	Arbitrary hopping matrix; <code>scale</code> required.

Sublattice-aware geometries ("honeycomb", "honeycomb_nnn", "kagome", "lieb", "dice") postpend a `sublattice_s` index of dimension n_{sub} (2 or 3) to the site chain. The physical site count becomes $n_{\text{sub}} \cdot 2^{L_x+L_y}$; `H.geometry` maps the enlarged 1-indexed site number to a real-space 2D position.

Cache and Truncation Helpers

Signature	Description
<code>_invalidate_cache!(H) → H</code>	Clear all cached results (<code>_tn_cache</code> , <code>_tn_mps_cache</code> , <code>_density_cache</code> , <code>scale/center</code>). Called automatically by all <code>add_*</code> functions.
<code>truncate!(H; cutoff=1e-10, maxdim=nothing) → H</code>	Truncate <code>H.mpo</code> in-place; invalidates all caches.

Incremental Hamiltonian Modification

All functions below mutate `H` in place and invalidate the Chebyshev cache. `add_hopping!` and `add_onsite!` must be called before `add_spin!` or `add_superconductivity!`.

The optional `position` keyword (default: inherits `H.aux_side`) controls whether auxiliary indices are prepended (`:pre`) or postpended (`:post`).

Signature	Description
<code>add_hopping!(H, f; maxdim=15, tol=1e-8, type=ComplexF64, apply_kwargs=NamedTuple())</code>	Add an arbitrary hopping $f(i, j)$ to <code>H.mpo</code> via QTCI. <code>apply_kwargs</code> is forwarded to the underlying <code>apply</code> call (e.g. (<code>maxdim=50</code> , <code>cutoff=1e-10</code> ,)). Errors if any auxiliary index is already present.
<code>add_onsite!(H, f; tol=1e-8)</code>	Add a diagonal on-site term $f(i)$ (1-indexed) via <code>get_diagonal_mpo</code> .
<code>add_spin!(H; cutoff=1e-8, maxdim=200, position=:pre)</code>	Prepend (or postpend) a spin- $\frac{1}{2}$ index: $\mathbf{I}_{\text{spin}} \otimes H$. No-op if already spinful.
<code>add_zeeman!(H, h; direction=:z, tol=1e-8, maxdim=200, position)</code>	Add Zeeman coupling $h \cdot S_\alpha$. <code>h</code> is a Number or function $h(i)$. <code>direction</code> $\in \{ :x, :y, :z \}$. Calls <code>add_spin!</code> automatically. If BdG is already applied, wraps in τ_z .
<code>add_superconductivity!(H, Δ; type=:swave, tol=1e-8, maxdim=200, position)</code>	BdG extension: prepend (or postpend) Nambu index. Spinless or singlet ($i\sigma_y \otimes \Delta$) depending on whether spin is present. <code>type=:swave</code> : $\Delta(i)$ diagonal; <code>type=:custom</code> : $\Delta(i, j)$ via TCI. Errors if called twice.
<code>add_soc!(H, λ; type=:rashba, direction=:z, tol=1e-8, maxdim=200, position)</code>	Spin-orbit coupling. <code>type=:rashba</code> : NN Rashba $\lambda(S_y K_u - S_y K_d)$ (scalar λ only). <code>type=:ising</code> : $\lambda(i) \cdot S_z$. <code>type=:custom</code> : arbitrary position MPO $\times$ spin operator given by <code>direction</code> .

Typical workflow:

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_onsite!(H, _ -> -0.5)           # chemical potential
add_zeeman!(H, 0.1)                 # Zeeman splits spin sectors
add_soc!(H, 0.05)                   # Rashba SOC
add_superconductivity!(H, 0.2)      # s-wave BdG
# H.sites = [nambu_s, spin_s, pos_qubits...] (default :pre stacking)
```

Geometry Helpers

Signature	Description
<code>honeycomb_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a honeycomb patch ($N = 2^L$).
<code>square_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a square lattice patch.
<code>triangular_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a triangular lattice patch. Odd rows offset by 0.5 in x ; $y = i_y \cdot \sqrt{3}/2$.
<code>triangular_bravais_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Positions for Bravais triangular lattice: $\mathbf{r}(i_x, i_y) = i_x \mathbf{a}_1 + i_y \mathbf{a}_2$ with $\mathbf{a}_1 = (1, 0)$, $\mathbf{a}_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$.
<code>honeycomb_sublattice_positions(Lx, Ly) → Matrix{Float64}</code>	$2N_{uc} \times 2$ positions for honeycomb with explicit sublattice: A at $(i_x + 0.5 i_y, i_y \sqrt{3}/2)$, B at $(\dots + 0.5, \dots + \sqrt{3}/6)$.
<code>kagome_positions(Lx, Ly) → Matrix{Float64}</code>	$3N_{uc} \times 2$ positions for the kagomé lattice (A, B, C sublattices).
<code>lieb_positions(Lx, Ly) → Matrix{Float64}</code>	$3N_{uc} \times 2$ positions for the Lieb lattice (A, B, C sublattices).
<code>dice_positions(Lx, Ly) → Matrix{Float64}</code>	$3N_{uc} \times 2$ positions for the dice (T3) lattice (hub A, rim B/C).
<code>lattice_positions(lattice, Lx, Ly; angle_deg=0.0) → Matrix{Float64}</code>	Positions for <code>:square</code> , <code>:triangular</code> , or <code>:honeycomb</code> , optionally rotated by <code>angle_deg</code> degrees about the centroid. (<code>twisted.tk.jl</code>)
<code>central_index(geom, N) → Int</code>	1-indexed site whose position is closest to the geometric centroid. <code>geom</code> is a callable <code>i -> pos_vec</code> . Errors if <code>H.geometry</code> is <code>nothing</code> .
<code>central_index(H) → Int</code>	Last L (or first L if <code>aux_side=:post</code>) entries of <code>H.sites</code> : position qubits, skipping all auxiliary indices.
<code>_pos_sites(H) → Vector</code>	

Low-Level MPO Builders (`Hamiltonian.jl`, `2D_lattice.jl`)

These functions build MPOs directly on a provided `sites` vector.

1D Kinetic Operators

Signature	Description
<code>kinetic_1d_nn(L, sites) → MPO</code>	Uniform NN hopping $\sum_i (i+1\rangle\langle i + \text{h.c.})$ via <code>sigma_plus/sigma_minus</code> binary increment/decrement operators.
<code>kinetic_1d_nn_custom(L, sites, hopping) → MPO</code>	Spatially varying NN hopping; <code>hopping</code> is a diagonal MPO of amplitudes.
<code>kineticNNN(L, sites, hopping, nn) → MPO</code>	1D chain with n -th-nearest-neighbour hopping and site-varying amplitude. <code>nn=1</code> = NN; larger <code>nn</code> built via <code>compose_power</code> .
<code>generate_kin_u(sites, N) → MPO</code>	Shift MPO $ n+1\rangle\langle n $ (binary increment / up-shift).

<code>generate_kin_d(sites, N) → MPO</code>	Shift MPO $ n-1\rangle\langle n $ (binary decrement / down-shift).
---	--

2D Kinetic Operators

Signature	Description
<code>intrachain_hopping(L_chain, N, sites; hopping=Id, t=1) → MPO</code>	NN x -direction hopping on a 2D lattice with row-break mask applied.
<code>interchain_hopping_square(L_chain, N, sites; hopping=Id, t=1) → MPO</code>	NN y -direction hopping on a square lattice.
<code>interchain_hopping_square_2nd_plus(L_chain, N, sites; hopping=Id, t2=1) → MPO</code>	NN hopping along $(+x, +y)$ diagonal (shift $+L_{\text{chain}} + 1$); row-break mask prevents wrap-around.
<code>interchain_hopping_square_2nd_minus(L_chain, N, sites; hopping=Id, t2=1) → MPO</code>	NN hopping along $(+x, -y)$ diagonal (shift $-(L_{\text{chain}} - 1)$).
<code>interchain_hopping_triangle(L_chain, N, sites) → MPO</code>	Both diagonal inter-row bonds on a triangular lattice (SW↗NE and SE↖NW).
<code>kineticintra2DNNN(Lx, Ly, sites, hopping, nn) → MPO</code>	Long-range intra-row hopping on $2^{L_x} \times 2^{L_y}$ square lattice. Row-boundary wrap suppressed by <code>_row_break_mpo</code> .
<code>kineticintra2DNNhex(Lx, Ly, sites, hopping, nn) → MPO</code>	Intra-row hopping for honeycomb; applies checkerboard A/B mask.
<code>kineticinterNNNSWNE(Lx, Ly, sites, hopping, nn) → MPO</code>	SW↗NE diagonal inter-row hopping (square lattice, <code>:xplus</code> mask).
<code>kineticinterNNNSENW(Lx, Ly, sites, hopping, nn) → MPO</code>	SE↖NW diagonal inter-row hopping (<code>:xplain</code> mask).
<code>kineticinterNNNtriSWNE(Lx, Ly, sites, hopping, nn) → MPO</code>	SW↗NE diagonal for triangular lattice; adds even-row selection mask.
<code>kineticinterNNNtriSEnw(Lx, Ly, sites, hopping, nn) → MPO</code>	SE↖NW diagonal for triangular lattice; adds odd-row selection mask.
<code>kineticinterNNNtri_bravais_diag(Lx, Ly, sites, hopping) → MPO</code>	Third-bond hopping for Bravais triangular lattice ($\Delta i_x = +1, \Delta i_y = -1$); mirrors SWNE logic with <code>kd/ku</code> swapped and <code>:xplus</code> break mask.

Diagonal Mask MPOs

Used internally as connectivity filters; all return diagonal 0/1 MPOs.

Signature	Description
<code>_row_break_mpo(Lx, Ly, sites; which::Symbol) → MPO</code>	Mask 0 at row boundaries. <code>which=:xplus</code> : last column; <code>:xplain</code> : first column.
<code>_row_select_mpo(_, Ly, sites; keep=:even) → MPO</code>	Retain only even or odd rows (<code>keep=:even</code> or <code>:odd</code> , 0-based).
<code>_col_select_mpo(Lx, Ly, sites; keep=:even) → MPO</code>	Retain only even or odd columns.
<code>_row_checker_mpo(Lx, Ly, sites) → MPO</code>	Checkerboard: 1 where $(i_x + i_y)$ is even, 0 otherwise.

<code>skeleton(L_chain, N, sites) → MPO</code>	0 where $i_x = 1$ (second column, 0-indexed), 1 elsewhere. Used internally for triangular wrap-around suppression.
--	--

QTCI-Based Builders

Signature	Description
<code>hopping2MPO(f, N, sites; tol=1e-8, initial_positions=[], type=Float64, unfoldingscheme=:interleaved) → MPO</code>	Compress $N \times N$ matrix $H[i, j] = f(i, j)$ into an MPO via QTCI. <code>unfoldingscheme=:interleaved</code> (default) or <code>:fused</code> .
<code>qtci_matrix_to_MPO(A.fun, L, sites; tol=1e-8, type=Float64) → MPO</code>	Like <code>hopping2MPO</code> for $2^L \times 2^L$ matrix functions; applies extra truncation at <code>maxdim=20</code> .
<code>get_diagonal_mpo(L, sites, f; type=Float64) → MPO</code>	Diagonal MPO with entry $f(x)$ at site $x \in \{1, \dots, 2^L\}$ (1-indexed). Compressed via 1D QTCI.
<code>qtt_mpo(L, xvals, sites, func; tol=1e-8, maxbonddim=50) → MPO</code> <code>compose_power(base, nn; side=:right, apply_kwarg) → MPO</code>	Compress scalar function <code>func(x)</code> on grid <code>xvals</code> into diagonal MPO via QTCI. base^{nn} by exponentiation-by-squaring in $O(\log n)$ applies. Returns identity if <code>nn=0</code> .

Preset Model Hamiltonians (direct MPO)

Function	Description
<code>HUniform(L, t; v=1e-6, nn=1)</code>	Uniform 1D chain with optional on-site potential <code>v</code> .
<code>HSSH(L, t, d; nn=1)</code>	SSH dimerized chain: hopping $t \pm d$ on alternating bonds.
<code>HAAH(L, V, phi, t; b=(1+√5)/2)</code>	Aubry–André–Harper: $H = -t \sum c_{i+1}^\dagger c_i + V \cos(2\pi b i + \phi) n_i$.
<code>HUniform2DSquare(Lx, Ly, t)</code>	Uniform square lattice tight-binding.
<code>HUniform2Dhex(Lx, Ly, t)</code>	Uniform honeycomb lattice (checkerboard intra-row mask).
<code>HUniform2Dtri(Lx, Ly, t)</code>	Uniform triangular lattice (three kinetic terms).
<code>HUniform2Dtri.bravais(Lx, Ly, t)</code>	Bravais triangular lattice: x, y , and diagonal bonds matching $\mathbf{a}_1 = (1, 0)$, $\mathbf{a}_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$.
<code>HChern8(Lx, Ly, V, t)</code>	8-fold Chern mosaic with spatially-varying hopping.
<code>H2DChernhex(Lx, Ly, t, t2, ms)</code>	Haldane Chern insulator on honeycomb; domain walls in t_2 and Semenoff mass.
<code>HQC2DSquare(Lx, Ly, t)</code>	Quasicrystal-modulated square lattice (8-fold wavevector pattern).
<code>build_hamiltonian(model, L; mparams, mparam_dict)</code>	Dispatch via <code>MODEL_REGISTRY</code> : build 1D MPO from model name and parameter string.
<code>build_hamiltonian(model, Lx, Ly; mparams, mparam_dict)</code>	2D variant.

Sublattice-based builders (return `TBHamiltonian` directly):

Function	Description
<code>honeycomb_sublattice_hamiltonian(Lx, Ly, t=1; cutoff, maxdim) → TBH</code>	Honeycomb NN Hamiltonian with 2-atom unit cell. 3 bond types: intra-cell A–B, x-intercell B→A, y-intercell B→A. <code>sublattice_s</code> postpended.
<code>honeycomb_nnn_hamiltonian(Lx, Ly, t=1, t2=0; cutoff, maxdim) → TBH</code>	Honeycomb NN+NNN: adds same-sublattice NNN hopping t_2 on top of NN t .
<code>kagome_hamiltonian(Lx, Ly, t=1; cutoff, maxdim) → TBH</code>	Kagomé lattice, 3-atom unit cell (A, B, C). <code>sublattice_s</code> dim-3 postpended.
<code>lieb_hamiltonian(Lx, Ly, t=1; cutoff, maxdim) → TBH</code>	Lieb lattice, 3-atom unit cell; has a flat band at $E = 0$. <code>sublattice_s</code> dim-3 postpended.
<code>dice_hamiltonian(Lx, Ly, t=1; cutoff, maxdim) → TBH</code>	Dice (T3) lattice: hub site A couples to rim sites B and C. <code>sublattice_s</code> dim-3 postpended.

Known models in MODEL_REGISTRY: "uniform", "ssh", "aah", "square_2d", "hex_2d", "triangular_2d", "triangular_bravais", "chern8", "chernhex", "qc2dsquare". Sublattice-aware geometries ("honeycomb", "honeycomb_nnn", "kagome", "lieb", "dice") are routed directly by `get_Hamiltonian` and are not in the plain MODEL_REGISTRY.

Generic n th-Neighbor Hopping (`nnneighbor_tk.jl`)

These functions operate on any 2D `TBHamiltonian` regardless of geometry, adding n th-nearest-neighbor bonds detected automatically via a reference patch.

Shell Detection

Signature	Description
<code>get_shell_disps(H, nn; Lx, Ly) → Vector{NTuple{4,Int}}</code>	Print and return the canonical displacement tuples $(\Delta i_x, \Delta i_y, s_{\text{from}}, s_{\text{to}})$ for the n th-nearest-neighbour shell. “Canonical” means $\Delta i_x > 0$, or $(\Delta i_x=0 \text{ and } \Delta i_y > 0)$, or $(\Delta i_x=\Delta i_y=0 \text{ and } s_{\text{from}} < s_{\text{to}})$. Call this first to discover which tuples will be passed to an amplitude function.

Hopping Accumulator

Signature	Description
<code>add_hopping_2D!(H, f; Lx, Ly, nn=1, maxdim=50, tol=1e-8) → H</code>	Add n th-nearest-neighbor hopping to H in place. Iterates over canonical shell displacements; for each, builds a forward+backward shift MPO weighted by the amplitude returned by <code>f</code> . Works for all 2D geometries including sublattice-aware ones (honeycomb, kagomé, Lieb, dice).

Amplitude function forms accepted by `f`:

Signature of f	Meaning
$f::\text{Number}$	Uniform amplitude for all bonds in the shell.
$f(\Delta ix, \Delta iy, fs, ts) \rightarrow \text{Number}$	Direction-dependent; receives canonical displacement and sublattice indices.
$f(n) \rightarrow \text{Number}$	Site-dependent (0-indexed unit-cell index n).
$f(ix, iy) \rightarrow \text{Number}$	Site-dependent via explicit 2D unit-cell coordinates.
$f(n, \Delta ix, \Delta iy, fs, ts) \rightarrow \text{Number}$	Both site (0-indexed) and direction.
$f(ix, iy, \Delta ix, \Delta iy, fs, ts) \rightarrow \text{Number}$	Full control: explicit 2D coords and direction.

Sublattice convention: For sublattice-aware geometries, fs/ts are 1-indexed sublattice numbers. The forward operator uses matrix element $M_{\text{fwd}}[s_{\text{from}}, s_{\text{to}}] = 1$, consistent with the original Hamiltonian builders.

Typical usage:

```
# Inspect which displacements will be passed (honeycomb, nn=2)
TensorBinding.get_shell_disps(H, 2; Lx=Lx, Ly=Ly)
# [(1,0,1,1), (0,1,1,1), (1,-1,1,1), (1,0,2,2), (0,1,2,2), (1,-1,2,2)]

# Uniform NNN hopping (same-sublattice, real amplitude)
add_hopping_2D!(H, 0.2; Lx=Lx, Ly=Ly, nn=2)

# Direction-dependent complex amplitude (Haldane-like phases)
add_hopping_2D!(H,
    (dx,dy,fs,ts) -> (dx,dy,fs,ts)==(1,0,1,1) ? t2*exp(im*phi) : 0.0im;
    Lx=Lx, Ly=Ly, nn=2)

# Position+direction dependent modulated hopping
add_hopping_2D!(H,
    (ix,iy,dx,dy,fs,ts) -> t1 + amp*sin(dot(G_vec[(dx,dy,fs,ts)], r_mid(ix,iy)));
    Lx=Lx, Ly=Ly, nn=1)
```

Bilayer and Multilayer Hamiltonians

Commensurate Stacking (bilayer_tk.jl)

Site encoding: site 1 = layer qubit (Index of dim n_{layers}); sites $2 \dots L + 1$ = position qubits. Constructors return a TBHamiltonian with $H.\text{layer_s}$ set.

Signature	Description
<code>bilayer_hamiltonian(lattice, Lx, Ly; stacking=:AA, t_intra=1.0, t_inter=0.3, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Two-layer Hamiltonian: $\sum_k P_k \otimes H_{\text{mono}} + (1\rangle\langle 2 + 2\rangle\langle 1) \otimes V$. stacking=:AA (on-site) or :Bernal (A_1 - B_2 , honeycomb only).
<code>multilayer_hamiltonian(lattice, Lx, Ly, n_layers; stacking=:AA, t_intra=1.0, t_inter=0.3, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Generalises to n_{layers} layers; interlayer coupling between adjacent pairs only.

<code>interlayer_mpo(lattice, stacking, Lx, Ly, sites; t_inter=1.0, cutoff=1e-8) → MPO</code>	Build the position-space interlayer coupling MPO V . Routes to <code>_bernal_interlayer_mpo</code> or <code>_aa_interlayer_mpo</code> .
<code>_bernal_interlayer_mpo(L, sites; t_inter=1.0, cutoff=1e-8) → MPO</code>	Exact Bernal A ₁ –B ₂ coupling: $V = t_{\perp}(K_u D_A + D_A K_d)$. D_A projects onto odd (A) sites (1-indexed); $K_{u/d}$ are ± 1 shifts.
<code>_aa_interlayer_mpo(sites; t_inter=1.0) → MPO</code>	AA stacking: on-site coupling $V = t_{\perp} \mathbf{I}$.

Twisted Multilayer (twisted.tk.jl)

Signature	Description
<code>twisted_multilayer_hamiltonian(lattice, Lx, Ly, angles_deg; t_intra=1.0, t_inter=0.3, α_decay=1/16, tol=1e-6, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Twisted multilayer: layer k rotated to <code>angles_deg[k]</code> . Interlayer coupling $V_{kl}(i, j) = t_{\perp} e^{-\alpha \mathbf{r}_k^{(i)} - \mathbf{r}_l^{(j)} }$ compressed via QTCI.
<code>twisted_bilayer_hamiltonian(lattice, Lx, Ly, θ_deg; ...) → TBHamiltonian</code>	Convenience wrapper: layer 1 at 0° , layer 2 at θ_{deg} .
<code>monolayer_hamiltonian(lattice, Lx, Ly, sites; t=1.0, cutoff=1e-8) → MPO</code>	Single-layer NN Hamiltonian MPO for <code>:square</code> , <code>:triangular</code> , or <code>:honeycomb</code> .

Layer-Core Prepend/Postpend Helpers

These functions extend an L -site MPO by one site by prepending or postpending a single-site layer operator. All are thin wrappers around `prepend_op/postpend_op`.

Signature	Description
<code>prepend_layer_projector(H, layer_s, k) → MPO</code>	Prepend $ k\rangle\langle k $ (1-indexed). Used for $P_k \otimes H$.
<code>prepend_layer_hopping(H, layer_s, k, l) → MPO</code>	Prepend $ k\rangle\langle l $. Used for interlayer $ k\rangle\langle l \otimes V$.
<code>postpend_layer_projector(H, layer_s, k) → MPO</code>	Postpend $ k\rangle\langle k $ to the end of H.
<code>postpend_layer_hopping(H, layer_s, k, l) → MPO</code>	Postpend $ k\rangle\langle l $ to the end of H.

Spin and Nambu Extensions (Supercond.tk.jl)

Index Constructors

Function	Description
<code>spin_index()</code>	Create a dim-2 Index tagged "Spin" (1= $\uparrow$, 2= $\downarrow$).
<code>nambu_index()</code>	Create a dim-2 Index tagged "Nambu" (1=particle, 2=hole).

Unified Prepend/Postpend Operators (utils.jl)

Signature	Description
<code>prepend_op(H, s, mat) → MPO</code>	General prepend: extends H by prepending an arbitrary matrix <code>mat</code> on site <code>s</code> .
<code>prepend_op(H, s, op::Symbol) → MPO</code>	Named operator lookup for "Spin" or "Nambu" tagged indices.
<code>prepend_op(H, s, k, l) → MPO</code>	Sparse $ k\rangle\langle l $ on site <code>s</code> (any dimension; 1-indexed).
<code>prepend_op(H, s, k) → MPO</code>	Projector $ k\rangle\langle k $.
<code>postpend_op(H, s, mat / op / k, l / k) → MPO</code>	Symmetric counterpart of <code>prepend_op</code> ; appends to the end of H .

Spin and Nambu Wrappers

Signature	Description
<code>prepend_spin(H, spin_s, op) → MPO</code>	Prepend a spin- $\frac{1}{2}$ operator. <code>op</code> is a <code>Symbol</code> or 2×2 matrix. Built-in ops: <code>:Id</code> , <code>:Pup</code> , <code>:Pdn</code> , <code>:Sz</code> , <code>:Sp</code> , <code>:Sm</code> , <code>:Sx</code> , <code>:Sy</code> , <code>:iSy</code> , <code>:miSy</code> .
<code>postpend_spin(H, spin_s, op) → MPO</code>	Postpend a spin- $\frac{1}{2}$ operator to the end of H . Same ops as above.
<code>prepend_nambu(H, nambu_s, op) → MPO</code>	Prepend a Nambu (particle-hole) operator. Built-in ops: <code>:Id</code> , <code>:Pp</code> , <code>:Ph</code> , <code>:tz</code> , <code>:tx</code> , <code>:ty</code> , <code>:tp</code> , <code>:tm</code> .
<code>postpend_nambu(H, nambu_s, op) → MPO</code>	Postpend a Nambu operator to the end of H . Same ops as above.

Higher-Level Assemblers

Signature	Description
<code>spin_hamiltonian(H_up, H_dn, spin_s; H_Zeeman=nothing, cutoff=1e-8) → MPO</code>	$P_{\uparrow} \otimes H_{\uparrow} + P_{\downarrow} \otimes H_{\downarrow} [+S_z \otimes H_{\text{Zeeman}}]$.
<code>bdg_hamiltonian(H_kin, H_pair, nambu_s; cutoff=1e-8) → MPO</code>	Spinless BdG: $\tau_z \otimes H_{\text{kin}} + \tau_+ \otimes H_{\text{pair}} + \tau_- \otimes H_{\text{pair}}^{\dagger}$.
<code>bdg_spin_hamiltonian(H_up, H_dn, H_pair, spin_s, nambu_s; H_soc=nothing, cutoff=1e-8) → MPO</code>	Spin- $\frac{1}{2}$ singlet BdG in Nambu-spin space. Pairing: $\tau_+ \otimes (i\sigma_y) \otimes H_{\text{pair}} + \text{h.c.}$. Optional Ising SOC $\tau_z \otimes S_z \otimes H_{\text{soc}}$.

Kernel Polynomial Method (KPM.tk.jl)

The KPM expands spectral quantities in Chebyshev polynomials $T_n(\tilde{H})$ where $\tilde{H} = (H - \text{center})/\text{scale} \in [-1, 1]$. Supported kernels: `:jackson` (default), `:lorentz`, `:fejer`, `:dirichlet`, `:hodc`.

Three complementary Chebyshev strategies are available, trading memory for generality:

- **MPO mode** (`mode=:mpo`): stores the full operator $T_n(\tilde{H})$ as an MPO. Needed for spectral functions at arbitrary sites, Green's functions, and density matrices. Bond dimension grows with n ; control with `maxdim`.

- **Diagonal/online mode:** extracts only the diagonal of each T_n as an MPS. Scales as $O(N_{\text{cheb}})$ MPO operations regardless of the number of energy points. Used by `get_ldos_spectrum`, `get_ldos_online`, and `get_ldos_spatial`.
- **MPS mode** (`mode=:mps`): propagates a reference state $|\phi_n\rangle = T_n(\tilde{H})|\psi_0\rangle$. Site-resolved LDoS at ψ_0 without storing any operator. Required for exciton systems (2L-site chain makes MPO mode prohibitive).

Spectral Bounds Estimation

Signature	Description
<code>_estimate_spectral_bounds(H_mpo, sites; dmrg_nsweeps=5, dmrg_maxdim=[10,20,40], dmrg_linkdim=4) → (scale, center)</code> <code>_ensure_scale!(H::TBHamiltonian; ...)</code>	Run DMRG to minimise H and $-H$, yielding $E_{\min}/E_{\max}$. Returns $\text{scale} = (E_{\max} - E_{\min})/2 \times 1.1$ and $\text{center} = (E_{\max} + E_{\min})/2$. If $H.\text{scale}==0$, call <code>_estimate_spectral_bounds</code> and store in $H.\text{scale}/H.\text{center}$. No-op otherwise. Called lazily by all high-level KPM functions.

Building the Chebyshev List

Signature	Description
<code>KPM_Tn(H_mpo, N, sites; scale=nothing, center=0.0, maxdim=40, cutoff=1e-8, ...) → (Tn_list, scale, center)</code> <code>KPM_Tn(H::TBHamiltonian, Ncheb; mode=:mpo, psi0=nothing, maxdim=40, cutoff=1e-8, ...) → (Tn_list, scale, center)</code>	Low-level MPO form. Build Chebyshev MPOs $T_n(\tilde{H})$ for $n = 0 \dots N$. If <code>scale=nothing</code> , bounds estimated via DMRG. High-level form. Calls <code>_ensure_scale!</code> lazily. <code>mode=:mpo</code> : caches MPO list in $H._\text{tn_cache}$. <code>mode=:mps</code> : requires <code>psi0</code> ; caches MPS list in $H._\text{tn_mps_cache}$. Both update $H._\text{tn_Ncheb}$.
<code>KPM_Tn(H::TBHamiltonian, Ncheb, X; maxdim=40, cutoff=1e-8, ...) → (Tn_mps_list, scale, center)</code>	Exciton convenience form. Builds MPS chain $T_n(\tilde{H}) X, X\rangle$ for 1-indexed exciton site X ; caches in $H._\text{tn_mps_cache}$. No MPO mode provided for 2L-site exciton systems.
<code>KPM_Tn_mps(H_mpo, N, psi0, sites; scale=nothing, center=0.0, maxdim=40, cutoff=1e-8, ...) → (Tn_mps_list, scale, center)</code> <code>KPM_Tn_mps(H::TBHamiltonian, N, psi0; maxdim=40, cutoff=1e-8, ...) → (Tn_mps_list, scale, center)</code>	Low-level MPS form. Builds $ \phi_n\rangle = T_n(\tilde{H}) \psi_0\rangle$ for $n = 0 \dots N$. Moments: $\mu_n = \langle \psi_0 \phi_n \rangle$. High-level MPS form. Calls <code>_ensure_scale!</code> ; caches in $H._\text{tn_mps_cache}$.

Important — return order: The return tuple is `(Tn_list, scale, center)`. A common bug is `Tn, _, scale = KPM_Tn(...)`, which assigns `center` to `scale`. The correct pattern is:

```
Tn, scale, center = KPM_Tn(H, Ncheb, sites)
omega_r = (omega - center) / scale # rescaled energy in (-1, 1)
```

When using the `TBHamiltonian` overload, `H.scale` and `H.center` are updated automatically and can be read directly.

Spectral Observables from Stored Chebyshev List

Signature	Description
<code>get_ldos_w_from_Tn(Tn_list, N, ω; maxdim=40, kernel=:jackson, ...) $\rightarrow$ MPO</code>	Full spectral MPO $\delta(\omega - H)$ at rescaled $\omega \in (-1, 1)$. Legacy full-MPO path; prefer <code>get_ldos</code> with <code>mode=:diag</code> for site-resolved LDOS.
<code>get_ldos_diag_from_Tn(Tn_list, N, ω_vals; kernel=:jackson, lambda=4.0, maxdim=40, cutoff=1e-8) $\rightarrow$ Vector{Union{Nothing,MPS}}</code>	Diagonal-MPS LDOS at all energies in a single pass over <code>Tn_list</code> . Returns one MPS $A(r, \omega)$ per energy (<code>nothing</code> for $ \omega \geq 1$).
<code>get_Green_retarded_from_Tn(Tn_list, N, ω; η=1e-2, maxdim=40, kernel=:jackson, ...) $\rightarrow$ MPO</code>	Retarded Green's function MPO $G^R(\omega + i\eta) = (\omega + i\eta - H)^{-1}$.
<code>get_density_from_Tn(Tn_list, N; fermi=0.0, maxdim=40, kernel=:jackson, ...) $\rightarrow$ MPO</code>	Density matrix $\rho = \theta(\mu - H)$ at rescaled Fermi level <code>fermi</code> .
<code>get_ldos_from_mun(mun_list, N, E; kernel=:jackson, lambda=4.0, ...) $\rightarrow$ Real</code>	Reconstruct LDoS at rescaled energy E from moment list $\mu_n = \langle \psi_0 T_n(\tilde{H}) \psi_0 \rangle$ (MPS mode output).
<code>get_ldos_hodc_from_mun(mun_list, N, E; eta=0.02, m_order=4) $\rightarrow$ Real</code>	HODC variant of <code>get_ldos_from_mun</code> ; sharper spectral features at higher cost.

High-Level LDOS Dispatch (TBHamiltonian interface)

Signature	Description
<code>get_ldos(H, ω_phys; mode=:diag, psi0=nothing, kernel=:jackson, lambda=4.0, eta=0.0, m_order=4, maxdim=40, cutoff=1e-8, zl, wl) $\rightarrow$ MPS or MPO or Real</code>	Unified LDOS dispatch. Requires a prior <code>KPM.Tn</code> call to populate the cache. <code>mode=:diag</code> (default): returns diagonal MPS (requires MPO cache). <code>mode=:mpo</code> : returns full spectral MPO (requires MPO cache; legacy). <code>mode=:mps</code> : returns <code>Real</code> from moments (requires MPS cache + <code>psi0</code>).
<code>get_ldos_spectrum(H, ω_phys_vals; kernel=:jackson, lambda=4.0, maxdim=40, cutoff=1e-8) $\rightarrow$ Vector{Union{Nothing,MPS}}</code>	Site-resolved LDOS at all energies in a single pass over the cached MPO list. Real-space analogue of <code>get_bands</code> . Requires MPO cache.
<code>get_ldos_online(H, Ncheb, X, ω_phys_vals; kernel=:jackson, lambda=4.0, maxdim=100, cutoff=1e-8, verbose=false) $\rightarrow$ Vector{Float64}</code>	Online LDoS at 1-indexed site X for all energies, using only 3 MPS simultaneously. Never stores any Chebyshev cache. Works for standard and exciton Hamiltonians (exciton: evaluation state is $ X, X\rangle$).

<code>get_ldos_spatial(H, Ncheb, ω_phys_vals; num_x=H.N, num_avg=1, mode=:mpo, x_start=1, x_end=H.N, x_groups=nothing, kernel=:jackson, lambda=4.0, maxdim=100, cutoff=1e-8, verbose=false) → Matrix{Float64}</code>	Spatially-resolved LDOS at <code>num_x</code> coarse positions, returning an $N_\omega \times N_x$ matrix. <code>mode=:mpo</code> : single online MPO pass, $O(N_{\text{cheb}})$ MPO ops regardless of <code>num_x</code> . <code>mode=:mps</code> : independent MPS recursion per position; cheaper for very large systems. <code>x_groups</code> : explicit <code>Vector{Vector{Int}}</code> for custom grouping. <code>num_avg</code> : sub-samples averaged per coarse position.
<code>get_dos_stochastic(H, Ncheb, ω_phys_vals; N_sample=50, N_bound=0, seed=42, kernel=:jackson, lambda=4.0, maxdim=100, cutoff=1e-8, verbose=false) → Vector{Float64}</code>	Stochastic full DoS via stratified random sampling. <code>N_sample</code> : random Hilbert-space samples; <code>N_bound</code> : dedicated bound-sector samples (exciton $ x, x\rangle$ states; 0 = uniform sampling). Combines with proper sector weights: $D \cdot \bar{A}_{\text{full}} + N_{\text{phys}} \cdot \bar{A}_{\text{bound}}$.

HODC Kernel

Signature	Description
<code>compute_hodc_params(m=6) → (z1, w1)</code>	Compute m quadrature nodes <code>z1</code> and weights <code>w1</code> for the HODC kernel. Call once; pass to KPM functions as <code>kernel=:hodc</code> , <code>z1=z1</code> , <code>w1=w1</code> .

Density Matrix Purification (Purification.tk.jl)

Iterative methods that drive eigenvalues of an approximate density matrix to exactly 0 or 1, refining the zero-temperature projector $\theta(\mu - H)$ without Gibbs ringing.

Initial Guess

Signature	Description
<code>purification_initial_guess(H_mpo, scale, sites; maxdim=40, cutoff=1e-8) → MPO</code>	Low-level form. Build $\rho_0 = \frac{1}{2}I - \frac{1}{2s}H$. Works only for center-symmetric spectra.
<code>purification_initial_guess(H::TBHamiltonian, maxdim=40, cutoff=1e-8) → MPO</code>	High-level form. Calls <code>_ensure_scale!</code> automatically. Uses formula accounting for nonzero center: $\rho_0 = (\frac{1}{2} + \frac{\text{center}}{2\text{scale}})I - \frac{H}{2\text{scale}}$.

McWeeny Purification

Signature	Description
<code>mcweeny_purify(ρ₀; maxiters=30, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	Iterate $\rho_{n+1} = 3\rho_n^2 - 2\rho_n^3$ until $\ \rho^2 - \rho\ /\ \rho\ < \text{tol}$. Quadratic convergence; 2 MPO products per step.

<code>mcweeny_purify(H::TBHamiltonian; maxiters=30, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	High-level form. Builds initial guess from H, runs McWeeny, caches result in <code>H._density_cache</code> .
---	--

SP2 Purification

Signature	Description
<code>sp2_purify(ρ_0, Nel; maxiters=40, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	Iterate: if $\text{Tr}(\rho^2) \geq N_{\text{el}}$: $\rho_{n+1} = \rho^2$; else $\rho_{n+1} = 2\rho - \rho^2$. Drives $\text{Tr}(\rho) \rightarrow N_{\text{el}}$; 1 MPO product per step.
<code>sp2_purify(H::TBHamiltonian; Nel=H.N÷2, maxiters=40, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	High-level form. Nel defaults to half-filling. Caches result in <code>H._density_cache</code> .

Quantum Fourier Transform and Band Structure (QFT_tk.jl)

QFT Conjugation

Signature	Description
<code>conjugate_by_qft(W; tol=1e-9, maxdim=100) → MPO</code>	Returns $UWU^\dagger$ where U is the normalised QFT MPO (LSB at site 1). Maps any real-space MPO to momentum space.

Online Band Structure: `get_bands`

get_bands runs an online Chebyshev KPM recurrence over the full MPO space without ever materialising the full T_n list. It replaces the legacy inner-product approach and requires no prior `KPM.Tn` call.

Signature	Description
<code>get_bands(H_mpo, scale, center, sites, Ncheb, D, ω_vals; ...) → Matrix{Float64}</code>	Low-level form. ω _vals must be in rescaled units $\in (-1, 1)$. D=1 or 2 sets the spatial dimension. Returns $N_\omega \times N_k$ matrix.
<code>get_bands(H::TBHamiltonian, Ncheb, D, ω_phys_vals; ...) → Matrix{Float64} or NamedTuple</code>	High-level form. Physical energies rescaled via <code>H.scale/H.center</code> . All auxiliary indices auto-detected from struct fields; no manual index passing. Returns <code>NamedTuple{Ak, ticks, labels}</code> when <code>kpath</code> kwarg is used, otherwise <code>Matrix{Float64}</code> .
<code>get_bands(H::TBHamiltonian, Ncheb, ω_phys_vals; ...) → Matrix{Float64} or NamedTuple</code>	Convenience form; infers D from <code>length(H.geometry(1))</code> . Errors if <code>H.geometry</code> is nothing.

Five-step projection pipeline (all steps optional and composable):

#	Step	Kwargs	Notes
0	Nambu (BdG)	<code>nambu_proj</code> ,	Sectors 1=particle, 2=hole.
1	Spin	<code>proj_nambu</code> <code>spin_proj</code> ,	Sectors 1= $\uparrow$, 2= $\downarrow$.
1c	Layer	<code>proj_s</code> , <code>spin_s_aux</code> <code>layer_proj</code> ,	For bilayer/multilayer models.
1b	Sublattice (aux)	<code>proj_layer</code> <code>sublat_proj</code> ,	For kagomé, Lieb, honeycomb, dice
2	Sublattice (legacy)	<code>proj_sl</code> , <code>sublat_s</code> <code>sublattice</code> ,	(<code>H.sublattice_s</code> set). Mask sandwich for preset models
		<code>proj_sl</code>	without <code>sublattice_s</code> .

`proj_*=nothing` sums over all sectors; `proj_*=k` selects sector k .

k-point sampling kwargs: `xmin`, `xmax`, `num_x` (grid in k_x); `ymin`, `ymax`, `num_y` (2D); `num_avg` (points averaged per center); `k_groups_override` (explicit path from `kpath_2d`).

High-symmetry k-path shortcut:

```
res = get_bands(H, Ncheb, 2, omega;
               kpath=[:G, :M, :Kp, :G], kpath_lattice=:honeycomb, num_x=30)
# res.Ak -- (N_omega x N_k) matrix
# heatmap(1:size(res.Ak,2), omega, res.Ak; xticks=(res.ticks, res.labels))
```

High-Symmetry k-Path Utilities

Signature	Description
<code>hsk_honeycomb(Lx, Ly) → NamedTuple</code>	High-symmetry k-points for the honeycomb sublattice Hamiltonian in quantic integer units. Fields: <code>G</code> =(0,0), <code>M</code> =($N_x/2, N_y/4$), <code>K</code> , <code>Kp</code> (Dirac points, rounded to nearest integer).
<code>hsk_square(Lx, Ly) → NamedTuple</code>	High-symmetry points for square lattice: <code>G</code> , <code>X</code> =($N_x/2, 0$), <code>M</code> =($N_x/2, N_y/2$).
<code>hsk_triangular(Lx, Ly) → NamedTuple</code>	Approximate high-symmetry points for triangular lattice: <code>G</code> , <code>M</code> , <code>K</code> .
<code>kpath_2d(hs_points, Lx; npts_per_segment=20) → (k_groups, tick_positions)</code>	Interpolate between (<code>kx_idx</code> , <code>ky_idx</code>) tuples; returns <code>k_groups</code> for <code>k_groups_override</code> and <code>tick_positions</code> for plot axes.
<code>kpath_setup(lattice, Lx, Ly, path_syms; npts_per_segment=20) → (k_groups, ticks, labels)</code>	Build k-path from symbol list (e.g. <code>[:G, :M, :Kp, :G]</code>) for given lattice (<code>:honeycomb</code> , <code>:square</code> , <code>:triangular</code>). Labels use Unicode: $G \rightarrow \Gamma$, $Kp \rightarrow K'$. Called internally by <code>get_bands</code> <code>kpath</code> shortcut.

Auxiliary Index Projection Helpers

Signature	Description
-----------	-------------

<code>project_aux(W, aux_s, sigma; side=:pre) → MPO</code>	Remove one auxiliary site from L -site MPO W by projecting onto state σ . Returns $(L - 1)$ -site position-only MPO. <code>side=:pre</code> (prepended) or <code>:post</code> (postpended).
<code>project_spin(W, spin_s, sigma) → MPO</code>	Convenience alias for <code>project_aux(W, spin_s, sigma; side=:pre)</code> .
<code>aux_site(H::TBHamiltonian, which) → (Index, Symbol)</code>	Return the auxiliary Index and its side (<code>:pre</code> / <code>:post</code>) for which $\in \{:\text{spin}, :\text{nambu}, :\text{layer}, :\text{sublattice}\}$. Auto-called by the <code>TBHamiltonian</code> overload of <code>get_bands</code> .

Note: `get_spect_k` and `get_spect_k_doubled` (inner-product approach) have been superseded by the diagonal-extraction approach in `get_bands` and are no longer part of the public API.

Topological Invariants (Topology_tk.jl)

Real-space topological invariants (Chern number and winding number) for general lattice geometries. All functions accept user-supplied coordinate functions `xfunc(i, L_chain)` and `yfunc(i, L_chain)` where i is a 0-indexed site number and `L_chain=2l` is the relevant lattice length.

Key concept — quenching: Periodic boundary conditions would make $\hat{x}$ discontinuous. With `quenched=true` the position operators are replaced by $\sin(x/\Lambda)$ and $\cos(x/\Lambda)$, which are bounded and smooth. The identity $x \approx \Lambda \sin(x/\Lambda)$ for small arguments means the quenched Chern marker recovers physical units via a factor Λ^2 . The winding-number density recovers via Λ .

Projector Retrieval

Signature	Description
<code>_get_projector(H; method=:KPM, fermi=0.0, Nchebychev=300, maxdim=40, cutoff=1e-8, Nel=nothing) → MPO</code>	Compute or retrieve the zero-temperature projector $P = \theta(\mu - H)$. <code>method=:KPM</code> : uses cached <code>H._tn_cache</code> or runs KPM; <code>fermi</code> in physical units. <code>method=:mcweeny</code> : uses <code>H._density_cache</code> or calls <code>mcweeny_purify</code> . <code>method=:sp2</code> : uses <code>H._density_cache</code> or calls <code>sp2_purify</code> with target <code>Nel</code> .

Quenched Position Operators

These build diagonal MPOs for the trigonometric quenching functions. The argument to `get_diagonal_mpo` is 1-indexed; these builders convert internally with `xfunc(i-1, L_chain)`.

Signature	Description
<code>get_sinx_op(L, sites, L_chain, Λ, xfunc) → MPO</code>	Diagonal MPO for $\sin(x_i/\Lambda)$.
<code>get_cosx_op(L, sites, L_chain, Λ, xfunc) → MPO</code>	Diagonal MPO for $\cos(x_i/\Lambda)$.

<code>get_siny_op(L, sites, L_chain, Λ, yfunc) → MPO</code>	Diagonal MPO for $\sin(y_i/\Lambda)$.
<code>get_cosy_op(L, sites, L_chain, Λ, yfunc) → MPO</code>	Diagonal MPO for $\cos(y_i/\Lambda)$.

1D Winding Number

Signature	Description
<code>get_W(H, xfunc, sz_func; method=:KPM, fermi=0.0, Nchebychev=300, maxdim=15, cutoff=1e-8, Nel=nothing, quenched=true, l=nothing, $\Lambda=10$) → Function</code>	Compute the winding-number density operator. Pre-computes one or two MPOs (W_1, W_2 for <code>quenched=true</code>). Returns a closure <code>calculate_winding(alpha::Int) -> ComplexF64</code> that evaluates the local winding density centred at site <code>alpha</code> (1-indexed).

Quenched case (`quenched=true`): Uses the 2-term decomposition $\langle \alpha | \hat{W} | \alpha \rangle = \Lambda [\cos(x_\alpha/\Lambda) \langle \alpha | W_1 | \alpha \rangle - \sin(x_\alpha/\Lambda) \langle \alpha | W_2 | \alpha \rangle]$ where $W_1 = \sigma_z(P \sin \hat{X} Q + Q \sin \hat{X} P)$, $W_2 = \sigma_z(P \cos \hat{X} Q + Q \cos \hat{X} P)$.

Flat case (`quenched=false`): builds $W = \sigma_z(P \hat{x} Q + Q \hat{x} P)$ once; the closure simply evaluates $\langle \alpha | W | \alpha \rangle$.

2D Real-Space Chern Marker

Signature	Description
<code>get_C_op_MPO_from_P(P, L, sites, xfunc, yfunc; l=nothing, $\Lambda=10$, maxdim=500, cutoff=1e-8, quenched=true) → Function</code>	Build real-space Chern marker from pre-computed projector P . Returns closure <code>calculate_chern_number(alpha::Int) -> ComplexF64</code> .
<code>get_C(H, xfunc, yfunc; method=:KPM, fermi=0.0, l=nothing, $\Lambda=10$, Nchebychev=300, maxdim=500, cutoff=1e-8, Nel=nothing, quenched=true) → Function</code>	High-level wrapper: calls <code>_get_projector</code> then <code>get_C_op_MPO_from_P</code> . Returns same <code>calculate_chern_number</code> closure.

Quenched case (`quenched=true`): uses the 4-term decomposition

$$\begin{aligned} \langle \alpha | \hat{C} | \alpha \rangle = 2\pi i \Lambda^2 & \left[\cos \frac{x_\alpha}{\Lambda} \cos \frac{y_\alpha}{\Lambda} \langle \alpha | C_1 | \alpha \rangle + \sin \frac{x_\alpha}{\Lambda} \sin \frac{y_\alpha}{\Lambda} \langle \alpha | C_2 | \alpha \rangle \right. \\ & \left. - \cos \frac{x_\alpha}{\Lambda} \sin \frac{y_\alpha}{\Lambda} \langle \alpha | C_3 | \alpha \rangle - \sin \frac{x_\alpha}{\Lambda} \cos \frac{y_\alpha}{\Lambda} \langle \alpha | C_4 | \alpha \rangle \right] \end{aligned}$$

where C_1 – C_4 are pre-computed α -independent MPOs. Only scalar trig evaluations are needed per site.

Flat case (`quenched=false`): $\hat{C} = 2\pi i (Q \hat{x} P \hat{y} Q - P \hat{x} Q \hat{y} P)$.

Thouless Pump

Computes the Thouless pump Chern number via the M1Q marker: $C = \text{M1Q}(T) - \text{M1Q}(0)$, where $\text{M1Q}(r, t) = \langle r | P(t) U^\dagger(t) \hat{x} U(t) P(t) | r \rangle$ and $U(t)$ is the adiabatic evolution operator propagated by $h(t) = [\dot{P}, P]$ via a second-order Taylor step.

Signature	Description
<code>get_pump_xop(L, sites, xfunc; quenched=false, $\Lambda=-1.0$) → MPO</code>	Build the position operator MPO for Thouless pump. quenched=false : diagonal entries are <code>xfunc(i,N)</code> directly. quenched=true : entries are $\Lambda \sin(x/\Lambda)$ to smooth the PBC discontinuity (Λ defaults to $N = 2^L$).
<code>thouless_pump(P_array, dt, x_op, sites; r_center, maxdim=100, cutoff=1e-8, verbose=false, return_trajectory=false) → Float64 or (Float64, Vector{Float64})</code>	Compute C from a pre-built array of projectors <code>P_array</code> (length <code>Nt</code>). <code>r_center</code> : 0-indexed bulk site at which M1Q is evaluated. With <code>return_trajectory=true</code> returns $(C, \text{M1Q_traj})$ where <code>M1Q_traj</code> has length <code>Nt+1</code> (index $1 = t = 0$).
<code>get_thouless_pump(H_of_t, Nt, T, xfunc; P_method=:mcweeny, fermi=0.0, Nchebychev=200, maxdim=100, cutoff=1e-8, quenched=false, $\Lambda=-1.0$, Nel=nothing, r_center=nothing, verbose=false) → Float64</code>	High-level wrapper. Calls <code>H_of_t(t_k)</code> to obtain the Hamiltonian at each of the <code>Nt</code> time steps, builds $P(t_k)$ via <code>P_method</code> (<code>:mcweeny</code> , <code>:sp2</code> , or <code>:KPM</code>), then calls <code>thouless_pump</code> . <code>r_center</code> defaults to $2^L/2$.

Typical usage:

```
# Factory: same site indices must be reused for all t
H0 = get_Hamiltonian("chain_1d", 1.0; L=8)
ref_sites = H0.sites
H_of_t(t) = get_Hamiltonian("ssh", (t=1.0, d=0.5*cos(2*pi*t/T));
                           L=8, ref_sites=ref_sites)

xfunc(i, N) = Float64(div(i, 2)) # x-coord for SSH (two sites per unit cell)
C = get_thouless_pump(H_of_t, 40, T, xfunc;
                     P_method=:mcweeny, verbose=true)
println("Thouless pump invariant: ", round(C; digits=3))
```

DMRG (DMRG_tk.jl)

Signature	Description
<code>dmrg_gs(H, sites; nsweeps=10, maxdim=[10,20,50,100,200], cutoff=1e-8, noise=[...], linkdim_init=10) → (E, ψ)</code>	Ground state and energy of H via DMRG.
<code>build_K(H_mpo, sites, ω, η; maxdim_K=200, cutoff_K=1e-8) → MPO</code>	Resolvent-squared MPO $K(\omega, \eta) = (H - \omega I)^2 + \eta^2 I$. Ground-state energy $E_K \geq \eta^2$; $E_K \rightarrow \eta^2$ at spectral peaks.

<code>dmrg_spectral(H_mpo, sites, ω, η; ψ_0=nothing, ...) $\rightarrow$ (E_K, ψ)</code>	DMRG ground state of $K(\omega, \eta)$. Output $ \psi(\omega)\rangle$ concentrates on the eigenstate of H nearest ω .
<code>local_weight(ψ, i, L, sites) $\rightarrow$ Float64</code>	$ \langle i \psi\rangle ^2$: LDoS proxy at site i (0-indexed, big-endian). Use with <code>dmrg_spectral</code> output.

Time Evolution (Timeev_tk.jl)

Propagator MPO and Single-Step TDVP

Signature	Description
<code>tdvp_evolve(H, ψ, dt; maxdim=200, cutoff=1e-10, normalize=true, nsite=2) $\rightarrow$ MPS</code>	Single TDVP step; thin wrapper around <code>ITensorMPS.tdvp</code> .
<code>build_tdvp_propagator_mpo(H, dt, L, sites; maxdim=50, cross_tol=1e-8, ...) $\rightarrow$ MPO</code>	Compress $U(\Delta t) = e^{-iH\Delta t}$ into an MPO via QTCI on matrix elements $\langle i U j\rangle$ evaluated by TDVP. Diagonal pivots seeded by default. <code>TBHamiltonian</code> overload available.
<code>apply_mpo_to_mps(U_mpo, ψ; cutoff=1e-12, maxdim=500, normalize=true) $\rightarrow$ MPS</code>	Apply propagator MPO to a single MPS state; optionally normalize.

Trajectory Evolution: MPS States

Signature	Description
<code>evolve_with_propagator(U_mpo, ψ_0, nsteps; normalize_each_step=true, cutoff=1e-8, maxdim=10000) $\rightarrow$ Vector{MPS}</code>	Apply the same propagator MPO <code>nsteps</code> times; returns length-(<code>nsteps</code> +1) trajectory including the initial state.
<code>evolve_with_tdvp(H, ψ_0, nsteps, dt; normalize_each_step=true, maxdim=200, cutoff=1e-10, ...) $\rightarrow$ Vector{MPS}</code>	Multi-step TDVP trajectory for time-independent Hamiltonian. Returns full trajectory [<code>psi(0)</code> , <code>psi(dt)</code> , ..., <code>psi(nsteps*dt)</code>]. <code>TBHamiltonian</code> overload: uses $-iH.mpo$.
<code>evolve_with_tdvp_timedep(Hoft, ψ_0, nsteps, dt; normalize_each_step=true, maxdim=200, cutoff=1e-10, reverse_step=false, krylovdim=20, tol=1e-10) $\rightarrow$ Vector{MPS}</code>	TDVP trajectory for a time-dependent $H(t)$. Uses midpoint rule: H is frozen at $H(t + \Delta t/2)$ over each interval. <code>Hoft</code> is a callable <code>t $\rightarrow$ MPO</code> (or <code>$\rightarrow$ TBHamiltonian</code>).

State Analysis Utilities

Signature	Description
<code>compute_basis_overlaps(states, L, sites) $\rightarrow$ NamedTuple</code>	Overlap of each MPS in <code>states</code> with all 2^L computational basis states. Returns (<code>overlaps</code> , <code>abs_overlaps</code> , <code>probabilities</code> , <code>norms</code>).

<code>basis_amplitude(ψ, n, L, sites) → ComplexF64</code>	Single-basis-state amplitude $\langle n \psi \rangle$ (0-indexed n).
<code>phase_aligned_distance(ψ_a, ψ_b) → Float64</code>	$\sqrt{2 - 2 \langle \hat{\psi}_a \hat{\psi}_b \rangle }$: distance minimised over global phase; 0 if states are equal up to phase and normalisation.
<code>check_tdvp_vs_U_mpo(H, U_mpo, dt, L, sites; ...) → (Float64, Float64)</code>	Validate <code>U_mpo</code> against direct TDVP on computational basis states. Returns (<code>max_overlap_error</code> , <code>max_distance</code>). TBHamiltonian overload available.

Density Matrix Dynamics (open / driven systems)

The density matrix ρ is represented as an MPO. The von Neumann equation $d\rho/dt = -i[H(t), \rho]$ is integrated with a classical RK4 scheme. Each $H(t)$ evaluation produces one MPO-MPO product, so bond dimension grows; use `maxdim` and `cutoff` to control cost.

Signature	Description
<code>rk4_step_dm_timedep(Hoft, ρ, t, dt; maxdim=200, cutoff=1e-10, truncate_intermediates=true) → MPO</code>	Single RK4 step for $d\rho/dt = -i[H(t), \rho]$. Evaluates H at the four RK4 sample times.
<code>evolve_rk4_dm_timedep(Hoft, ρ_0, nsteps, dt; maxdim=200, cutoff=1e-10, verbose=false) → Vector{MPO}</code>	Full RK4 trajectory; returns length-(<code>nsteps</code> +1) list including ρ_0 .
<code>dm_expect(O, ρ) → Float64</code>	Expectation value $\text{Tr}(O\rho)$ for Hermitian O . Uses <code>inner(O, rho)</code> .
<code>purity(ρ) → Float64</code>	$\text{Tr}(\rho^2) = \text{inner}(\rho, \rho)$. 1 for pure states, < 1 for mixed.
<code>purity_trajectory(states) → Vector{Float64}</code>	Apply purity to each MPO in <code>states</code> .

Observables Along Trajectories

Signature	Description
<code>observables_trajectory(ops, states::Vector{MPO}) → Dict</code>	Evaluate a named collection of MPO observables on a density-matrix trajectory. <code>ops</code> is a <code>DictString,MPO</code> ; returns <code>DictString,VectorFloat64</code> .
<code>timedep_observable_trajectory(Oft, states::Vector{MPO}, dt) → Vector{Float64}</code>	Evaluate a time-dependent observable <code>Oft(t) → MPO</code> at times $t_k = k \cdot \Delta t$ on the density-matrix trajectory.
<code>bond_current_x(ρ, j, tx, L, sites) → ComplexF64</code>	Bond current $J_j = 2 \text{Im}[t_x \rho_{j,j+1}]$ on the bond $(j, j+1)$ (0-indexed j). Uses off-diagonal matrix elements of ρ via <code>matrix_checker</code> .
<code>bond_current_x_trajectory(states, j, tx, L, sites; dt=1.0) → Vector{ComplexF64}</code>	Evaluate <code>bond_current_x</code> at every step of a trajectory.

<code>central_x_bond(L; Nx=nothing) → Int</code>	Return the 0-indexed site index of the central x -bond. For 2D ($L_x = L_y = L/2$) provides the horizontal bond nearest the geometric center; uses L_x rows if Nx is given.
--	---

Diagnostic Tools

Signature	Description
<code>compare_propagator_and_tdvp_heatmaps(U_mpo, ψ_0, L, sites, nsteps; ...) → NamedTuple</code>	Run both propagator-MPO and TDVP trajectories from the same initial state and return basis-state probability heatmaps for side-by-side comparison. TBHamiltonian overload available.

Two-Particle Systems (twoparticle_tk.jl)

Constructs the exciton (electron-hole) Hamiltonian on an interleaved $2L$ -site space.

Site encoding: L position qubits per sector, interleaved as $[e_1, h_1, e_2, h_2, \dots]$: odd sites $(1, 3, \dots)$ = electron, even sites $(2, 4, \dots)$ = hole. The Hamiltonian is $H_{\text{exc}} = H_c \otimes I_h - I_e \otimes H_v + U$ where U is a contact interaction diagonal in the electron-hole coincidence basis.

TBHamiltonian.L = position qubits per sector; TBHamiltonian.sites holds all $2L$ interleaved sites; TBHamiltonian.N = 2^L (single-sector physical sites).

High-Level Constructors

Signature	Description
<code>exciton_hamiltonian(geometry, params, Ufunc; L, on_site=nothing, scale=nothing, tol_quantics=1e-8, maxbonddim_quantics=100, tol=1e-8, cutoff=1e-8, maxdim=200, kwargs...) → TBHamiltonian</code>	Build an exciton Hamiltonian where both electron and hole sectors share the same geometry/params. Extra kwargs are forwarded to get_Hamiltonian.
<code>exciton_hamiltonian(H_c, H_v, Ufunc; on_site=nothing, scale=nothing, tol_quantics=1e-8, maxbonddim_quantics=100, tol=1e-8, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Two-sector form for distinct electron/hole bands. H_c and H_v must have the same L and distinct site indices.

Keyword arguments (both forms):

- `Ufunc(x)` — contact interaction strength at site $x \in \{1, \dots, 2^L\}$ (1-indexed).
- `on_site` — conduction-band-edge modulation $V(x)$; added as $+V$ on electron, $-V$ on hole (type-I confinement: both carriers confined). Compressed via QTCL.
- `scale` — exciton half-bandwidth; 0.0 triggers lazy DMRG estimation on first KPM call.

Low-Level MPO Builders

Signature	Description
<code>Exciton_Hamiltonian(H_c, H_v, Ufunc; on_site, tol_quantics, maxbonddim_quantics, tol, cutoff, maxdim) → MPO</code>	Low-level builder returning a raw $2L$ -site MPO. Interleaves electron/hole kinetic terms and adds contact interaction.
<code>hopping_mpo_exciton(H_c, H_v) → MPO</code>	Embed single-particle sectors into the $2L$ -site space: $H_c \otimes I_h - I_e \otimes H_v$ without interaction.
<code>build_interaction_op_exciton(L, sites, Ufunc) → MPO</code>	Build the contact interaction diagonal MPO. $Ufunc(x)$ on the diagonal where electron and hole coincide ($x_e = x_h = x$); off-diagonal elements are zero.

Typical usage:

```
# 1D exciton -- same geometry, contact interaction
H_exc = exciton_hamiltonian("chain_1d", 1.0, x -> -0.5;
                           L=8, on_site = x -> -0.1*exp(-(x-64)^2)/200))

# 2D exciton -- separate electron/hole bands
H_c = get_Hamiltonian("square_2d", 1.0; L=8, Lx=4, Ly=4)
H_v = get_Hamiltonian("square_2d", 0.5; L=8, Lx=4, Ly=4)
H_exc = exciton_hamiltonian(H_c, H_v, x -> -0.2)

# Use with standard TensorBinding tools
Tn, scale, center = KPM_Tn(H_exc, 200)
dos = real(tr(get_ldos_w_from_Tn(Tn, 200, 0.0; maxdim=200)))
```

Random Phase Approximation (RPA_tk.jl)

Computes the RPA susceptibility χ^{RPA} via the Dyson equation $(1 - \Pi_0 V)\chi = \Pi_0$, where Π_0 is the Lindhard (bubble) function.

Pipeline

1. `get_Tnlists` — build Chebyshev lists for system 1, system 2, and the effective combined Hamiltonian $H_{\text{eff}} = \mathbf{I} \otimes H_2 - H_1 \otimes \mathbf{I}$.
2. `get_bubble_expanded_from_Tn` — compute $\Pi_0(\omega)$ as a $2L$ -site MPO.
3. `build_bubble_mpo` — collapse to an L -site MPO.
4. `rpa_from_bubble_diag` — solve the Dyson equation via DMRG linear solver.

Functions

Signature	Description
<code>get_Tnlists(H, H2, sites, sites2, N; a=6, maxdim=100) → (Tn1, Tn2, Tn_eff)</code>	Build 3 Chebyshev lists: individual and effective.

<code>get_bubble_expanded_from_Tn(Tn1, Tn2, Tn_eff, s1, s2, N, ω, ε_F; a=6, maxdim=200) → MPO</code>	Lindhard bubble $\Pi_0(\omega)$ as a $2L$ -site MPO.
<code>build_bubble_mpo(ω; Tn1, Tn2, Tn_eff, sites, sites2, finalsites, finalfinalsites, ...) → MPO</code>	Collapse $2L$ -site bubble to L -site MPO.
<code>rpa_from_bubble_diag(Π, V, finalsites, finalfinalsites; nsweeps=20, maxdim=400, cutoff=1e-8) → MPS</code>	Solve $(I - \Pi_0 V)\chi = \Pi_0$ via DMRG linear solver.

Tensor Utilities (used internally by RPA)

Signature	Description
<code>mpo_kron(A, B) → MPO</code>	Concatenate two MPOs (Kronecker product on site space).
<code>mps_kron(A, B) → MPS</code>	Concatenate two MPS.
<code>replace_sites(MPO, newsites) → MPO</code>	Remap physical indices of an MPO to <code>newsites</code> , preserving prime levels.
<code>interleave_mpo(mpo, phys_sites, n) → MPO</code>	Embed L -site MPO into $2L$ -site space by interleaving with identity.
<code>collapse_mpo_pairs(mpo2L, out_sites) → MPO</code>	Merge consecutive site pairs $(2n-1, 2n)$ to single site: $2L \rightarrow L$.
<code>extract_diagonal_to_mps(M) → MPS</code>	Extract diagonal of an MPO as an MPS by projecting bra=ket.
<code>merge_mps_to_mpo(mps) → MPO</code>	Contract consecutive pairs of an MPS into MPO tensors.
<code>apply_interleave_swaps(W, sites; cutoff=1e-16, maxdim=200) → MPO</code>	Reorder sites from block order $[1 \dots N, N+1 \dots 2N]$ to interleaved via SWAP gates.

Self-Consistent Mean Field (Meanfi.tk.jl)

Signature	Description
<code>SCF_Hubbard(L, sites, U, k_mpo, max_iter, N, threshold, mpo_guess_up, mpo_guess_dn, mps_guess_up, mps_guess_dn, qtt_den_old_up, qtt_den_old_dn, mix)</code>	Hartree-Fock SCF loop for the Hubbard model. Iterates $H_{\uparrow} = H_{\text{kin}} + U\rho_{\downarrow}$ and vice versa until $\ \rho_{\text{new}} - \rho_{\text{old}}\ < \text{threshold}$. Accepts linear mixing parameter <code>mix</code> . Returns <code>(qtt_new_up, qtt_new_dn, Tn_up, Tn_dn, conv_error)</code> .

Utilities (utils.jl)

Signature	Description
<code>to_binary_vector(n, L) → Vector{String}</code>	Convert integer n to length- L big-endian binary string vector.

<code>binary_to_MPS(n, L, sites) → MPS</code>	Computational-basis state $ n\rangle$ as an L -site MPS.
<code>matrix_checker(mpo, sites, i, j) → Number</code>	Matrix element $\langle i \text{mpo} j \rangle$ by contracting basis MPS (small systems only). Optional <code>L</code> argument accepted but ignored.
<code>get_matrix(mpo, sites) → Matrix</code>	Full $D \times D$ dense matrix of <code>mpo</code> (feasible for $D \lesssim 512$). Optional <code>L</code> argument accepted but ignored. Works for any site types.
<code>fix_sites(mpo, sites) → MPO</code>	Replace site indices of an MPO (e.g. from TCI) with target <code>sites</code> .
<code>custom_mpo(mps, new_sites) → MPO</code>	Convert a $2N$ -site MPS (interleaved QTCI) into an N -site MPO.
<code>fused_mpo(mps, new_sites) → MPO</code>	Convert an N -site MPS with dim-4 physical indices (fused grid) into an N -site MPO.
<code>mps2mpo(L, sites, density_mps) → MPO</code>	Promote a diagonal MPS to a diagonal MPO via <code>_asdiagonal</code> .
<code>custom_mps(qtt, sites) → MPS</code>	Replace site indices of a TCI MPS with target <code>sites</code> .
<code>build_shift_mpo(sites, q) → MPO</code>	Cyclic shift MPO: maps $ x\rangle \rightarrow x+q\rangle \pmod{2^L}$ on a binary-encoded chain. Built site-by-site using binary addition with carry; <code>q</code> is an integer offset.
<code>mpsexciton(x, sites) → MPS</code>	Exciton basis state $ x, x\rangle$ on the interleaved $2L$ -site electron-hole chain. <code>x</code> is 1-indexed; odd sites = electron qubit, even sites = hole qubit.
<code>mps_to_diagonal_mpo(mps, sites) → MPO</code>	Convert an MPS to a diagonal MPO on <code>sites</code> by replacing each physical index with a bra-ket pair tied by a delta tensor. Used to embed QTCI-compressed 2D functions (e.g. from <code>build_interaction_op_exciton</code>) into MPO form.

Custom Operator Extensions

Defined in `utils.jl` and `2D_lattice.jl`:

Name	Site type	Matrix
<code>sigma_plus</code>	Qubit	$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ — binary increment (up-shift)
<code>sigma_minus</code>	Qubit	$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$ — binary decrement (down-shift)
<code>sigma_d</code>	Qubit	$\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ — projector onto $ 1\rangle$
<code>sigma_u</code>	Qubit	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ — projector onto $ 0\rangle$

Built-in $\text{spin-}\frac{1}{2}$ operators (used by `prepend_spin/postpend_spin`):

Symbol	Matrix	Symbol	Matrix
:Id	$\mathbf{I}_2$	:Sz	$\frac{1}{2}\sigma_z = \text{diag}(\frac{1}{2}, -\frac{1}{2})$
:Pup	$\text{diag}(1, 0)$	:Sp	$ \uparrow\rangle\langle\downarrow $
:Pdn	$\text{diag}(0, 1)$	:Sm	$ \downarrow\rangle\langle\uparrow $
:Sx	$\frac{1}{2}\sigma_x$	:Sy	$\frac{1}{2}\sigma_y$
:iSy	$i\sigma_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	:miSy	$-i\sigma_y$

Built-in Nambu operators (used by `prepend nambu/postpend nambu`):

:Id	$\mathbf{I}_2$	:tz	$\tau_z = \text{diag}(1, -1)$
:Pp	$\text{diag}(1, 0)$	:tp	$ p\rangle\langle h $
:Ph	$\text{diag}(0, 1)$	:tm	$ h\rangle\langle p $
:tx	σ_x	:ty	σ_y

Quick-Reference Workflows

KPM Density of States and Band Structure

```
# 1. Build Hamiltonian (scale=0 means lazy DMRG estimation)
H = get_Hamiltonian("square_2d", 1.0; L=8)

# 2. Build Chebyshev list (caches in H._tn_cache / H.scale / H.center)
Tn, scale, center = KPM_Tn(H, 200; maxdim=100)

# 3. Sweep in energy (always rescale before calling KPM functions)
omega_r = (omega_phys - center) / scale # must be in (-1, 1)

# 4. Observables
A = get_ldos_w_from_Tn(Tn, 200, omega_r; maxdim=100) # delta(omega-H)
dos = real(tr(A)) # density of states
G = get_Green_retarded_from_Tn(Tn, 200, omega_r; eta=0.02)
rho = get_density_from_Tn(Tn, 200; fermi=omega_r_fermi)

# 5. Band structure A(k,omega) -- works for plain, spinful, BdG, layered H
Akw = get_bands(H, omega_phys_vals) # trace over aux DOF
Akw = get_bands(H, omega_phys_vals; aux_proj=[1]) # spin-up only
```

Density Matrix Purification

```
# McWeeny: one-liner (initial guess built and cached internally)
rho_mc = mcweeny_purify(H; maxiters=30, maxdim=200, tol=1e-5, verbose=true)

# SP2: one-liner with explicit electron count
rho_sp = sp2_purify(H; Nel=H.N div 2, maxiters=40, maxdim=200)

# Low-level form (for non-standard center or external MPO):
rho0 = purification_initial_guess(H; maxdim=100, cutoff=1e-8)
rho = mcweeny_purify(rho0; maxiters=30, maxdim=200)
```

Topological Invariants

```
# Define coordinate functions (0-indexed site i, 0-indexed length L_chain)
```

```

xfunc(i, L) = Float64(i % Int(sqrt(L))) # x-coordinate on square lattice
yfunc(i, L) = Float64(i div Int(sqrt(L))) # y-coordinate
sz_func(i) = isodd(i) ? 1.0 : -1.0 # sublattice / sigma_z (1-indexed)

# 1D winding number (returns a closure; pre-computes 2 MPOs once)
winding = get_W(H, xfunc, sz_func; quenched=true, Lambda=10.0, maxdim=40)
nu = sum(real(winding(alpha)) for alpha in 1:H.N) / H.N

# 2D Chern marker (returns a closure; pre-computes C1--C4 MPOs once)
chern = get_C(H, xfunc, yfunc; quenched=true, Lambda=10.0, maxdim=500)
C_local = real(chern(H.N div 2)) # evaluate at central site
C_avg = sum(real(chern(a)) for a in 1:H.N) / H.N

```

Bilayer and Twisted Multilayer

```

# Bernal bilayer graphene (exact interlayer, no TCI)
H_bg = bilayer_hamiltonian(:honeycomb, 4, 3;
    stacking=:Bernal, t_intra=1.0, t_inter=0.3)
# H_bg.sites = [layer_s, pos_qubits...]

# Twisted bilayer at 5 degrees
H_tbg = twisted_bilayer_hamiltonian(:triangular, 4, 4, 5.0;
    t_intra=1.0, t_inter=0.3, alpha_decay=1/16)

```

Spinful Superconductor (BdG)

```

H = get_Hamiltonian("square_2d", 1.0; L=8)
add_onsite!(H, _ -> -2.0) # chemical potential
add_zeeman!(H, 0.1; direction=:z) # spin-splitting
add_soc!(H, 0.05; type=:rashba) # Rashba SOC
add_superconductivity!(H, 0.3) # singlet pairing
# H.sites = [nambu_s, spin_s, pos_qubits...] (aux_side=:pre)
# H.N is unchanged; the MPO now spans 2*2*N = 4N states

# Band structure tracing over Nambu and spin:
Akw = get_bands(H, omega_phys_vals)
# Band structure in the particle sector only:
Akw = get_bands(H, omega_phys_vals; aux_proj=[1, 1])

```

Band Structure with Sublattice Resolution

```

# Sublattice-aware honeycomb (H.sublattice_s is set)
H = get_Hamiltonian("honeycomb", (t=1.0,); L=8, Lx=4, Ly=4)
KPM_Tn(H, 200) # not needed -- get_bands runs online
Ak = get_bands(H, 200, 2, omega_vals) # sum over sublattices
Ak_A = get_bands(H, 200, 2, omega_vals;
    sublat_proj=true, proj_sl=1) # sublattice A only

# High-symmetry path (G--M--K'--G for honeycomb)
res = get_bands(H, 200, 2, omega;
    kpath=[:G, :M, :Kp, :G], kpath_lattice=:honeycomb, num_x=40)
heatmap(1:size(res.Ak,2), omega, res.Ak; xticks=(res.ticks, res.labels))

```

```
# Legacy sublattice projection (models without H.sublattice_s, e.g. "hex_2d")
H2 = get_Hamiltonian("hex_2d", 1.0; L=8)
Ak = get_bands(H2, 200, 2, omega_vals; sublattice=true, proj_sl=1)
```

aux_side: postpending auxiliary DOF

By default (`aux_side=:pre`) auxiliary indices are prepended. To instead append them, pass `position=:post` to the first extension function:

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_spin!(H; position=:post)           # H.sites = [pos_qubits..., spin_s]
add_superconductivity!(H, 0.1)        # H.sites = [pos_qubits..., spin_s, nambu_s]
# All subsequent add_* calls inherit H.aux_side = :post
```

Krylov Green's Function (`krylov_tk.jl`)

Computes the retarded single-particle Green's function $G(\omega) = (\omega + i\eta - H)^{-1}$ as an MPO by solving the vectorized linear system $[(\omega + i\eta - H) \otimes I] |G\rangle = |I\rangle$ via `ITensorMPS.linsolve` (a DMRG-like Krylov solver). No KPM Chebyshev expansion is required; the unscaled Hamiltonian is used directly.

Signature	Description
<code>get_green_krylov(H_mpo, sites, ω_{phys}; $\eta=1\text{e-}2$, nsweeps=12, maxdim=100, cutoff=1e-8, x0_mpo=nothing, ishermitian=false, tol=1e-10, maxiter=600, krylovdim=30, verbose=false) $\rightarrow$ MPO</code>	Low-level form. Solve on raw MPO <code>H_mpo</code> .
<code>get_green_krylov(H::TBHamiltonian, ω_{phys}; ...) $\rightarrow$ MPO</code>	High-level form. Forwards to the MPO overload using <code>H.mpo</code> and <code>H.sites</code> .

Key kwargs:

- `x0_mpo`: optional MPO warm-start (e.g. a cheap KPM Green's function). Vectorized via `_vec_mps_from_mpo` and passed as initial guess to `linsolve`.
- `ishermitian`: set `true` only when the shifted operator is truly Hermitian (not physical for retarded GF with real $\eta > 0$).
- `krylovdim`: Krylov subspace dimension per site (default 30).

Usage:

```
G = get_green_krylov(H, omega; eta=0.05, nsweeps=20, maxdim=200)
dos = -imag(tr(G)) / pi
ldos = real(inner(psi_i, apply(G, psi_i))) # LDoS at site i

# Warm-start from KPM estimate
G_kpm = get_Green_retarded_from_Tn(H._tn_cache, H._tn_Ncheb, (omega-H.center)/H.scale; eta)
G      = get_green_krylov(H, omega; x0_mpo=G_kpm, nsweeps=8, maxdim=300)
```

Flake Masking (Flake_tk.jl)

Restricts a Hamiltonian to an irregular geometry by applying a smooth diagonal mask $M_i = \sigma(\text{sdf}(\mathbf{r}_i)/\sigma_0)$, where $\text{sdf}(\mathbf{r})$ is a signed-distance function (positive inside the flake) and σ is the logistic sigmoid. The resulting $H_{\text{flake}} = M H M$. Call `mask_hamiltonian` *before* `add_spin!`, `add_superconductivity!`, or any bilayer constructor.

SDF Primitives

Signature	Description
<code>sdf_interval(x_lo, x_hi) → x → Float64</code>	1D interval $[x_{\text{lo}}, x_{\text{hi}}]$; positive inside.
<code>sdf_disk(cx, cy, r) → (x,y) → Float64</code>	Disk of radius <code>r</code> centred at <code>(cx,cy)</code> ; positive inside.
<code>sdf_rect(cx, cy, w, h) → (x,y) → Float64</code>	Axis-aligned rectangle of full width <code>w</code> , height <code>h</code> ; positive inside.
<code>sdf_halfplane(nx, ny, d) → (x,y) → Float64</code>	Half-plane $n_x x + n_y y \geq d$; <code>(nx,ny)</code> need not be unit-length.
<code>sdf_annulus(cx, cy, r_in, r_out) → (x,y) → Float64</code>	Circular annulus; implemented as <code>sdf_subtract(disk_out, disk_in)</code> .
<code>sdf_convex_polygon(vertices) → (x,y) → Float64</code>	Convex polygon from CCW vertex list of <code>(x,y)</code> tuples.

CSG Combinators

Function	Description
<code>sdf_union(f, g)</code>	Boolean union: $\max(f, g)$; positive where either region holds.
<code>sdf_intersect(f, g)</code>	Boolean intersection: $\min(f, g)$; positive inside both.
<code>sdf_subtract(f, g)</code>	Subtraction: $\min(f, -g)$; removes <code>g</code> from <code>f</code> .

Mask Application

Signature	Description
<code>mask_hamiltonian(H, sdf; sigma=0.3, tol=1e-8, maxdim=200, cutoff=1e-8) → TBHamiltonian</code>	Apply smooth flake mask to <code>H</code> : learns M_i via QTCI, returns $H_{\text{flake}} = M H M$ as a new <code>TBHamiltonian</code> . <code>sigma</code> : boundary half-width in lattice units; smaller <code>sigma</code> = sharper edge.

Examples:

```
H = get_Hamiltonian("square_2d", 1.0; L=8, Lx=4, Ly=4)
```

```
# Disk
```

```
H_disk = mask_hamiltonian(H, sdf_disk(8.0, 8.0, 6.0); sigma=0.3)
```

```
# Triangular flake (intersection of three half-planes)
```

```
tri = sdf_intersect(sdf_intersect(
```



```
sdf_halfplane(0, 1, 2.0),  
sdf_halfplane(-sqrt(3)/2, -0.5, -8.0)),  
sdf_halfplane( sqrt(3)/2, -0.5, -8.0))  
H_tri = mask_hamiltonian(H, tri; sigma=0.2)  
  
# Ring  
H_ring = mask_hamiltonian(H, sdf_annulus(8.0, 8.0, 3.0, 7.0); sigma=0.3)
```